

● دلوقتي النظام كامل بقى شكله كده:

User (GUI)
↓
Frontend (HTML/JS)
↓ HTTP Request
FastAPI (Backend)
↓ Query
Elasticsearch
↓ Results
Backend
↓ JSON
Frontend
↓ Display
User sees results

● أهم Ports عندك

Service	URL	Port
FastAPI	http://127.0.0.1:8000	8000
GUI (Live Server)	http://127.0.0.1:5500	5500
Elasticsearch	http://localhost:9200	9200

التشغيل النهائي

(1) شغلي Elasticsearch

```
docker run -p 9200:9200 -e "discovery.type=single-node" elasticsearch:8.11.0
```

(2) اعملي indexing

داخل Backend:

```
python index_files.py
```

FastAPI شغلي (3)

```
python -m uvicorn main:app --reload
```

(4) افتحي الـ GUI

لو عندك `index.html` :

```
frontend/index.html
```

اختبار الـ API مباشرة

Search

```
http://127.0.0.1:8000/search?q=search
```

Filter by filetype

```
http://127.0.0.1:8000/search?q=search&filetype=pdf
```

Stats

```
http://127.0.0.1:8000/stats
```

أول حاجة: المشروع ده إيه أصلاً؟ 🧠

📌 Final Project: Mini Search Engine

يعني المطلوب منك عملي محرك بحث صغير زي Google بس بسيط جدًا.

♦ وظيفته:

- يقرأ ملفات من فولدر (folder)
- يخزن محتواها بطريقة ذكية (indexing)
- وبعد كده يسمح للمستخدم يعمل search داخل الملفات دي

📁 1. النظام هيشغل على ملفات إيه؟

المحرك لازم يقدر يقرأ:

- TXT
- PDF
- JSON
- CSV
- Excel (.xlsx)

⚙️ 2. قبل ما يبدأ يعمل index

👉 المستخدم لازم يختار:

"أنا عايز أفهرس إيه؟"

مثال:

- أختار TXT + PDF فقط
- أو أختار Excel + CSV

💡 المهم:

إنت اللي تقرر UI إزاي (checkbox / CLI / config file)

😬 3. سؤال مهم جدًا (JSON / CSV / Excel)

المطلوب منك تفكري:

مثلاً:

- لوحده؟ object كل الملف كنص؟ ولا كل JSON:
- CSV: row document كل؟ ولا الملف كله؟
- Excel: sheet ولا cell؟

المهم: 

- تختاري طريقة
- وتكتبها في المشروع (Documentation)

4. Features (أهم جزء) البحث

المستخدم لازم يقدر يعمل search بـ كل الأنواع دي:

4.1 Boolean Search

يعني:

- AND → لازم الكلمتين موجودين
- OR → أي كلمة
- NOT → استبعد كلمة

مثال:

```
java AND python
java OR python
java NOT python
```

4.2 Grouping

باستخدام الأقواس:

```
(java OR python) AND database
```

4.3 Phrase Search

جملة كاملة بين quotes:

```
"information retrieval"
```

4.4 Fuzzy Search (تصحيح أخطاء الكتابة)

لو كتبت غلط:

```
retrival~
```

أو مع درجة خطأ:

```
retrival~2
```

4.5 Wildcard Search

•

- = أي حاجة
- ? = حرف واحد

مثال:

```
inform*  
?earch
```



4.6 Filter by Date

تقدري تعملي فلتر على الملفات حسب:

- تاريخ التعديل

باستخدام:

```
os.path.getmtime()
```

4.7 Filter by File Type

مثلاً:

- أبحث في PDF بس
- أو TXT بس

4.8 Highlighted Snippets

يعني لما يظهر نتيجة:

- يبين جزء من النص
- ويعمل highlight للكلمة اللي اتبحث عنها

4.9 Did you mean?

لو المستخدم كتب غلط أو مفيش نتائج:

Did you mean: retrieval?

4.10 Paging (صفحات النتائج)

- كل صفحة فيها 5 نتائج

- Next / Previous

5. Statistics (إحصائيات النظام)

المستخدم لازم يقدر يشوف:

1. عدد الملفات المفهرسة

Total documents: 120

2. تقسيم حسب النوع

pdf: 40
txt: 50
csv: 20

3. Top 10 كلمات

أكثر كلمات مستخدمة في الملفات

6. شكل الواجهة (UI)

إنّ حرية تختاري:

- CLI (terminal)
- GUI (desktop)
- Web app (browser)

💡 المهم: أي UI لازم يدعم كل الوظائف دي

7. أهم حاجة في UI لازم يعملها المستخدم

المستخدم لازم يقدر:

1. يختار file types

2. يعمل build index

3. يشوف summary بعد build

4. يكتب search query

5. يعمل date filter (اختياري)

6. يعمل file type filter (اختياري)

7. يشوف النتائج (اسم الملف + النوع + snippet + date + score)

8. Did you mean

9. يقلب بين الصفحات

10. يفتح stats

⚠ أهم فكرة في المشروع كله

المشروع ده مش مجرد search

هو 3 حاجات مع بعض:

1. Indexing (فهرسة الملفات)
2. Query Parser (فهم البحث)
3. Ranking + UI (عرض النتائج بشكل ذكي)

1) Pick which file formats to include in the index

دي معناها:

المستخدم يحدد أنواع الملفات اللي عايز يدور فيها.

إنت عاملها بالـ checkbox ✓

✓ Test 1 — نوع واحد بس

اعمل:

✓ فقط PDF

وابحث:

search

المتوقع:

كل النتائج تكون:

pdf

بس.

✓ أكثر من نوع — Test 2

اعملني ✓:

- TXT
- JSON

وابحثني:

data

المتوقع:

النتائج تكون:

- txt
- json

فقط.

✓ بدون اختيار أي نوع — Test 3

شيلي كل الـ checkboxes.

وابحثني:

information

المتوقع:

يجيب كل الأنواع عادي:

- pdf
 - txt
 - csv
 - json
 - xlsx
-
-

● 2) Type any search query

يعني يدعم:

- Boolean
 - Phrase
 - Fuzzy
 - Wildcard
-

✓ Boolean Query

جربي:

search AND engine

المتوقع:

لازم الملف يحتوي:

- search
- engine

الالتنين معًا.

✓ OR Query

search OR data

المتوقع:

أي ملف يحتوي:

- search
أو
- data

✓ NOT Query

search NOT engine

المتوقع:

يجيب الملفات اللي فيها:

✓ search

لكن:

✗ بدون engine

✓ Phrase Query

"information retrieval"

المتوقع:

لازم الجملة كاملة بنفس الترتيب.

مش مجرد:

- information
- retrieval

متفرقين.

✓ Fuzzy Query

retrival~

أو:

retrival~2

المتوقع:

حتى لو الكلمة غلط:

retrieval

يرجع نتائج.

✓ Wildcard Query

نجمة *

inform*

المتوقع:

يجيب:

- information
- informative
- informing

علامة استفهام ?

?earch

المتوقع:

يجيب:

- search

لأن:

s + earch

3) Filter by date range and file type

✓ File Type Filter

✓ اختاري:

TXT

وابحثي:

data

المتوقع:

كل النتائج:

txt

بس.

✓ Date Range Filter

اعملي:

Start Date = 2026-05-01

End Date = 2026-05-10

وابحثي أي كلمة.

المتوقع:

النتائج تكون لملفات اتعديت فقط في الفترة دي.

✓ Date + Type مع بعض

✓ PDF

وحددي تاريخ.

المتوقع:

يجيب:

- فقط pdf

- داخل التاريخ فقط

● 4) Highlighted snippets + Paging

✓ Highlight Test

ابحثي:

search

المتوقع:

الكلمة نفسها تبقى متهايلايت:

مثلا:

This file contains `<em>search</em>` engine concepts

✓ مهم جدًا

المفروض:

✗ مش الجملة كلها خضرا

✓ الكلمة فقط.

✓ Paging Test

لازم يكون عندك أكثر من 5 نتائج.

ابحثي كلمة شائعة:

data

المتوقع:

الصفحة الأولى:

5 نتائج فقط.

وبعدين:

Next

يجيب الصفحة اللي بعدها.

⚠️ لو مفيش Next/Previous Buttons

يبقى الـ paging في الـ backend موجود،

لكن الـ frontend ناقص.

● 5) Did You Mean

✓ Test

ابحثي:

retrivalll

المتوقع:

بدل:

No results

يظهر:

Did you mean: retrieval

⚠️ لو بيقول:

No results found

بس،

يبقى الـ suggestion logic لسه ضعيف شوية.

● 6) Stats About Index

افتحي: ✓

http://127.0.0.1:8000/stats

المتوقع:

يرجع JSON فيه:

Total Documents

```
"total_documents": 20
```

Breakdown By Type

```
"file_types": [  
  {  
    "key": "pdf",  
    "doc_count": 5  
  }  
]
```

Top Terms

```
"top_terms": [  
  {  
    "key": "search",  
    "doc_count": 10  
  }  
]
```

-
- أيوه، ده بالطبط اللي كنا بنعمله فوق 🏆

إحنا وصلنا لفكرتين:

1) وقت الـ Indexing

لما اليوزر يختار:

- PDF
- TXT

وبعدين يدوس:

Build Index

فالـ backend يعمل:

```
if ext not in selected_types:
    continue
```

وده معناه:

✓ الملفات دي بس اللي تتخزن في Elasticsearch أصلاً.

وده يحقق requirement:

pick which formats to index before building

(2) وقت الـ Search

إنتي عندك checkboxes وقت السيرش.

فالبيوزر ممكن يقول:

- دور في PDF بس
- أو TXT + JSON

وده بيعمل:

terms

وقت البحث filter

وده:

✓ Search-time filtering

فإحنا كنا بنخلي المشروع يعمل الاتنين

المرحلة	الوظيفة
Build Index	اختيار إيه يتخزن أصلاً
Search	فلتره من اللي متخزن

وده أحسن implementation فعلاً للمشروع.

1) paging 2 لازم 5 مش

في الـ backend:

```
size = 5  
from = (page - 1) * 5
```

وفي frontend:

```
let pageSize = 5;
```